

Image to Garment

3D Vision Project SS 26

Prerana Achar, Siddharth Pandey

July 31, 2026

1 Motivation

Reconstructing digital garments from a single RGB image is relevant to applications such as virtual try-on, digital fashion, animation, and game development. However, a generic 3D reconstruction is insufficient for many applications. An unstructured mesh or another 3D representation can reproduce the garment in the observed pose, but it does not indicate how the garment was reconstructed, which makes it difficult to edit, resize, retarget to a different body, or simulate under new poses. Sewing patterns provide a more suitable representation for such tasks. They describe the intrinsic rest shape of a garment through 2D fabric panels and the seams connecting them. Once recovered, these panels can be modified by editing their parameters and can also be draped on a body using physical simulation. We use GarmentCode [3] to facilitate the generation of sewing patterns. It defines a procedural sewing-pattern program whose continuous parameter vector decodes into a manufacturable, simulation-ready pattern. GarmentCode formalizes this as a domain-specific language for constructing parametric sewing patterns. Recovering such a representation from a single image is nevertheless challenging because of ambiguities arising from factors such as camera viewpoint, material properties, illumination, and occlusions. Recent work approaches this problem by fine-tuning a large vision-language model to emit these parameters, assuming that the task requires the semantic reasoning of a language model.

In this project, we try to identify whether we can map a single image to GarmentCode parameters without a VLM and avoid its computational costs. Given an RGB image, our model predicts the categorical garment structure and the continuous parameters associated with its components. We test whether a task-specific, topology-conditioned predictor can learn the dependency between topology and numerical parameters within a more constrained output space than general-purpose language generation. We train our model on synthetic GarmentCode data generated by ChatGarment [1], mixed with our own dataset. We use a frozen encoder (DINOv2) [5] to produce a feature vector for the image, which is then passed through a layer of MLPs to produce the final set of parameters, resulting in a clean and editable reconstruction. Furthermore, we fine-tune a large language model to enable garment editing. It is trained on parameter and text inputs and outputs the edited garment parameters. Our code can be found here: [GitHub](#).

2 Prior Work

Early learning-based garment sewing-pattern reconstruction methods relied on access to 3D garment geometry. NeuralTailor [2] predicts variable-cardinality sewing-pattern structures from garment point clouds using a unified model that processes multiple garment types and generalises to pattern topologies not observed during training. Later, SewFormer [4] used a single image for reconstruction. It introduced the SewFactory dataset, containing one million rendered images paired with ground-truth sewing patterns, and proposed a two-level transformer for predicting garment panel structure and geometry. ChatGarment uses a fine-tuned VLM to output GarmentCode parameters. Among existing approaches, ChatGarment is most closely related to our setting. It fine-tunes LLaVA, a large vision-language model, to produce a structured JSON description containing garment categories and continuous design parameters. The description is then converted into sewing patterns using an extended GarmentCode system. The use of a VLM enables it to support reconstruction from images, generation from text, and language-based garment editing. Our project studies a narrower question: whether we can use a simpler model while improving the results produced by ChatGarment.

3 Data

3.1 GarmentCode

GarmentCode [3] is a procedural sewing-pattern generator. It takes a design-configuration JSON file, a nested tree of named parameters, and produces flat 2D garment panels, which can then be stitched and draped on a body to produce a 3D mesh. Our goal is to infer these design parameters from a single RGB photograph of a garment. The advantage of using this parameter space is that it produces clean geometry: the output is, by construction, a manufacturable garment; i.e., the panels close, the seams match, and the result can be resimulated on a different body or edited by a human designer. This kind of clean geometry is not possible with current 3D generation methods.

GarmentCode’s JSON contains three types of parameters:

- Continuous parameters: These are continuous numeric values normalised to the range 0–1. Example: ‘collar.width = 0.358’.
- Numerical categories: These are categorical values that are also normalised to 0–1 and have numerical relationships between them. Example: ‘collar.component.depth = 7’.
- Categorical parameters: These include other categorical values of type string, Boolean, or ‘null’. Example: ‘meta.bottom = ‘PencilSkirt’’, ‘sleeve.sleeveless = false’.

Because different garments have different structures—e.g., a pencil skirt has no flare parameters, and a sleeveless top has no cuff parameters—we do not define a separate parameter set for each garment type. Instead, the schema is the *union* of every parameter observed in the dataset, and

each parameter is assigned a fixed index. Any individual garment then activates only the subset of indices belonging to its own structure, and we record this in a binary *activity mask*.

An important property of this mask is that it is not independent information: it is fully determined by the categorical parameters. A slot is active only because a type selector, such as `meta.bottom` or `sleeve.sleeveless`, has switched that branch of the tree on. The model therefore never predicts the mask directly. At training time, the true mask is available from the labels; during inference on an unlabelled image, it follows from the *predicted* categorical parameters.

3.2 ChatGarment Dataset

We use part of the ChatGarment dataset for garment diversity [1]. Each procedurally generated garment is rendered on a body in several pose frames, and each frame is photographed from four camera positions. It contains almost 9,000 garments across several bodies from the BEDLAM dataset [7].

3.3 Our Dataset

We generate 600 unique garments across 100 body types. Each garment is draped on a body textured using the BEDLAM dataset, and three independent views are then rendered with an image-generation model (GPT-Image-2). Since GarmentCode has many parameters, a classifier trained on an unbalanced dataset could cheat by ignoring the tails. To prevent this, our data-generation process creates a balanced dataset by ensuring that all parameters are well covered.



Figure 1: Visual comparison of samples from our dataset and the ChatGarment dataset.

4 Methodology

4.1 Garment Reconstruction

Given a single RGB image, we first pass it through a frozen encoder, DINOv2 (22M), to obtain an image feature vector. The encoding is then passed to one of the prediction architectures: a two-head MLP used as a baseline, a three-head region-specific MLP, a parameter-wise MLP, or an ensemble. Each architecture predicts the complete GarmentCode schema: the categorical, continuous, and ordinal parameters. During training, the prediction losses are evaluated only for parameters that are active in the corresponding ground-truth garment. During inference, the predicted parameters are combined into a valid GarmentCode configuration file (JSON/YAML) and decoded into two-dimensional sewing patterns along with a body-draped 3D reconstruction. The loss for each MLP model is described below, whereas the ensemble model has its own loss, which is described in the respective section.

$$\mathcal{L}_{\text{shared}} = \mathcal{L}_{\text{num}} + 0.45 \mathcal{L}_{\text{cat}} + 0.20 \mathcal{L}_{\text{ord}}$$

The numeric loss is a SmoothL1/Huber function on continuous numeric outputs, the categorical loss is a label-smoothed cross-entropy function for categorical outputs, and the ordinal loss ensures that the models correctly order numerical categorical values.

$$\mathcal{L}_{\text{num}} = \frac{\sum_i m_i w_i \rho_{0.04}(\hat{y}_i - y_i)}{\sum_i m_i w_i^{\text{num}}}$$

$$\rho_{\beta}(x) = \begin{cases} \frac{1}{2} \frac{x^2}{\beta}, & |x| < \beta, \\ |x| - \frac{1}{2}\beta, & \text{otherwise,} \end{cases} \quad \beta = 0.04$$

Here i is the numeric output index. m_i is the mask checking whether a numeric output is present in the ground-truth JSON, and w_i is the weight for a particular numeric entry based on the number of available samples.

$$\mathcal{L}_{\text{cat}} = \frac{\sum_i m_i w_i^{\text{cat}} \left[(1 - \epsilon) (-\log p_{i,y_j}) + \epsilon \left(-\frac{1}{C_i} \sum_{c=1}^{C_i} \log p_{i,c} \right) \right]}{\sum_i m_i w_{i,y_i}^{\text{cat}}}$$

Here, $\epsilon = 0.10$; $p_{i,c}$ is the predicted softmax probability for class c of categorical field i ; and C_i is the number of classes in field i .

$$\mathcal{L}_{\text{ord}} = \frac{\sum_i m_i w_{i,y_i}^{\text{cat}} \sum_{c=1}^{C_i} p_{i,c} |v_{i,c} - v_{i,y_i}|}{\sum_i m_i w_{i,y_i}^{\text{cat}}}$$

Here, $v_{i,c}$ is the normalised ordinal value of class c in field i .

4.1.1 Two-head MLP

This is the simplest baseline model. It passes the DINOv2 image encodings to two separate MLP heads. The numerical head jointly predicts all continuous and ordinal numerical parameters, while the categorical head produces the packed logits for every categorical field in the GarmentCode schema. The outputs are unpacked according to the fixed schema indices before the corresponding losses are evaluated.

4.1.2 Three-head MLP

In this model, we incorporate garment-specific knowledge by splitting the output space into three categories instead of making one MLP head work for the entire garment. The frozen encoder produces an embedding that is passed through a shared layer before being split into three branches by body region:

Branch	Width	Covers
Upper body	128	Shirt, collar, sleeve, asymmetry
Lower body	128	Skirts and trousers
Waistband	64	Waistband parameters

Each branch is then split into its own regression and classification slices, which are later packed back into a common schema layout. We test the hypothesis that the parameters within a body region are correlated with one another and largely independent across regions, so partitioning the parameters along these lines should help the region-specific head generalise better.

4.1.3 Multi-MLP

The Multi-MLP architecture assigns an individual prediction head to every GarmentCode parameter. The DINOv2 embedding is first passed through a shared feature transformation, after which many small parameter-specific MLPs independently predict their respective outputs. Each continuous parameter head produces one scalar, while each categorical head produces logits over the valid choices for that field. Continuous predictions are passed through a sigmoid and rescaled to the valid range of the corresponding GarmentCode parameter.

4.1.4 Ensemble models

The motivation for the ensemble model was that each of the above MLPs, although performing well on the continuous variables, tended to overfit the categorical variables because of the limited diversity in the training data.

It uses one frozen DINOv2 patch encoder and then sends the extracted patch and class-token features to several GarmentExpertHead members. Each expert predicts the garment topology

(route), which is then used to generate categorical and numerical values. The model also includes route constraints over topology roots such as the upper garment, waistband, and bottom garment, which help prevent invalid GarmentCode combinations, such as activating skirt fields for a trouser route. The numerical output values are also conditioned on this topology. Final predictions are produced by aggregating the ensemble members with learned stacker weights, allowing the system to combine multiple expert opinions while enforcing a valid garment structure.

The losses used for this model are:

$$\mathcal{L}_{\text{ensemble}} = \mathcal{L}_{\text{shared}} + 0.35 \mathcal{L}_{\text{router}} + 0.15 \mathcal{L}_{\text{pose-consistency}}$$

The shared losses are the same as those used for the other models. The router loss ensures that ensemble members do not select different routes for the same garment, such as predicting trousers for a skirt garment. They must all agree on the topology. The pose-consistency loss ensures that when models receive different subsets of the data containing images of the same garment in different poses, the generated garment remains consistent across the heads.

$$\mathcal{L}_{\text{router}} = \text{CE}_{\epsilon}(z, r) + \frac{1}{|\mathcal{R}|} \sum_{j \in \mathcal{R}} \text{CE}_{\epsilon}(z^{(j)}, y^{(j)})$$

Here, z represents the root topology logits (whether the garment has this topology field), and r is the ground-truth target tuple. $\mathcal{R}$ is the set of all possible routes, while $z^{(j)}$ and $y^{(j)}$ are the logits and ground-truth values for the actual topology class.

$$\begin{aligned} \mathcal{L}_{\text{pose}} = & \text{SmoothL1}_{\beta=0.02}(\hat{x}^{(1)}, \hat{x}^{(2)}) + \\ & 0.25 \cdot \frac{1}{|\mathcal{A}|} \sum_{c \in \mathcal{C}} \text{JSD}(p_c^{(1)}, p_c^{(2)}) + \\ & 0.05 \cdot \left[1 - \text{mean}_i \cos(f_i^{(1)}, f_i^{(2)}) \right] \\ \text{JSD}(p^{(1)}, p^{(2)}) = & \frac{1}{2} D_{\text{KL}}(p^{(1)} \parallel \bar{p}) + \frac{1}{2} D_{\text{KL}}(p^{(2)} \parallel \bar{p}), \quad \bar{p} = \frac{1}{2} (p^{(1)} + p^{(2)}) \end{aligned}$$

Jensen-Shannon Divergence (JSD) makes the categorical values agree with each other and the cosine distance makes the numerical values similar.

4.2 Editing Garments

In addition, we fine-tune Qwen2.5-3B-Instruct, a large language model, to enable garment editing. It takes an existing GarmentCode configuration together with a text prompt, such as "make the sleeves longer", as input and predicts the corresponding edited configuration. We fine-tune it using 4-bit QLoRA, with rank-16 adapters applied to the attention and feed-forward projection layers. Consequently, only roughly 31 million parameters are updated for each variant. The goal is to make the model edit only the required parameters and leave the others unchanged. The garment configuration is serialised as text, and the language model autoregressively predicts its structure and categorical values. Since large language models are not good at emitting precise continuous values, we handle the prediction of continuous floating-point values through separate regression heads.

We implement two ways of predicting continuous values. In the single-token variant, following the implementation of ChatGarment [1], every continuous value in the target configuration is replaced by a placeholder, and a single `<ALLNUM>` token is added at the end of the numerical section. The hidden state at this token is then passed through an MLP that predicts the complete 76-dimensional numerical vector in a predetermined order.

In the per-token variant, every active continuous value is replaced by an individual `<VAL>` token preceded by its parameter key. A shared MLP maps each `<VAL>` hidden state to one scalar, so the number of numerical predictions varies with the number of active parameters in the garment. This was implemented so that each prediction has direct context about the parameter it represents and avoids using a single hidden-state vector to predict the complete 76-dimensional output.

The MLP heads consist of two layers with layer normalisation, a 512-dimensional hidden layer with GELU activation, and a linear output layer. In both variants, optimisation uses a joint textual and numerical objective. Autoregressive cross-entropy is applied only to the target configuration tokens, with the input prompt excluded from the loss. An L1 loss supervises the continuous predictions, giving the total objective as follows:

$$\mathcal{L} = \mathcal{L}_{\text{text}} + 0.1 \times \mathcal{L}_{\text{num}}$$

5 Results

5.1 Garment Reconstruction

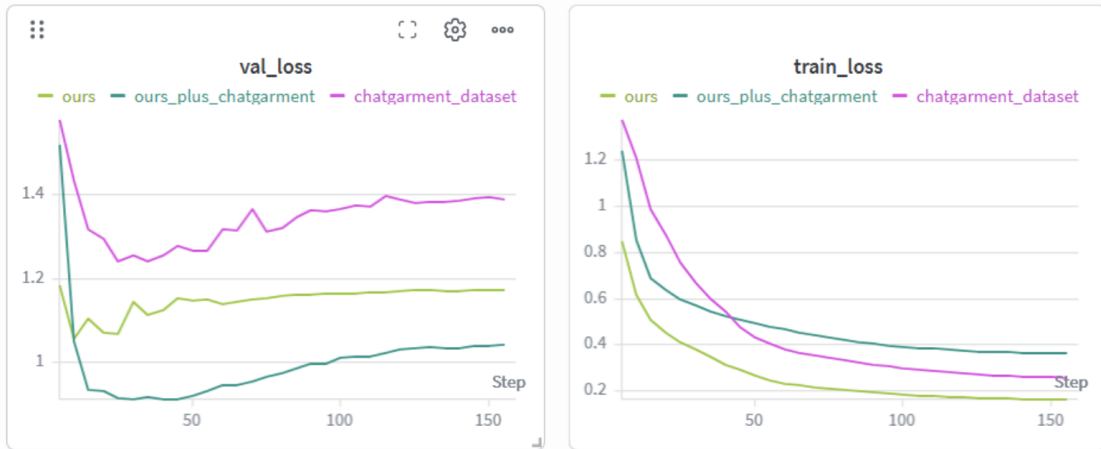


Figure 2: Training and validation losses for the different dataset configurations.

The figure depicts that the model trained on the combined dataset achieves the lowest validation loss, showing that adding ChatGarment samples improves generalisation despite producing a higher training loss. Training only on our smaller dataset gives the lowest training loss but a higher validation loss, indicating overfitting, while the ChatGarment-only split performs worst on validation.

Model	Trainable parameters	Train loss	Val loss	L2 loss	Eval time (s)	Time/image (s)
Two-head MLP	592,255	1.0932	0.9813	–	1359	13.32
Multi-MLP	16,213,582	1.1441	0.9826	–	1377	13.50
Three-head MLP	410,815	1.1071	0.9739	–	1505	14.75
Ensemble fixed-v2	3,925,425	0.4892	0.6763	–	1701	16.68
ChatGarment	312,799,308	–	–	–	2633	25.81

Table 1: Comparison of prediction head variants against ChatGarment

The ensemble achieves the lowest training and validation losses, clearly outperforming the three MLP baselines. Among the simpler models, the three-head MLP gives the best validation loss while using the fewest trainable parameters, suggesting that grouping outputs by body region is effective. The ensemble is slower than the MLP variants but remains substantially faster and much smaller than ChatGarment.

The following are comparisons of our predictions with ChatGarment’s predictions on several in-the-wild images.

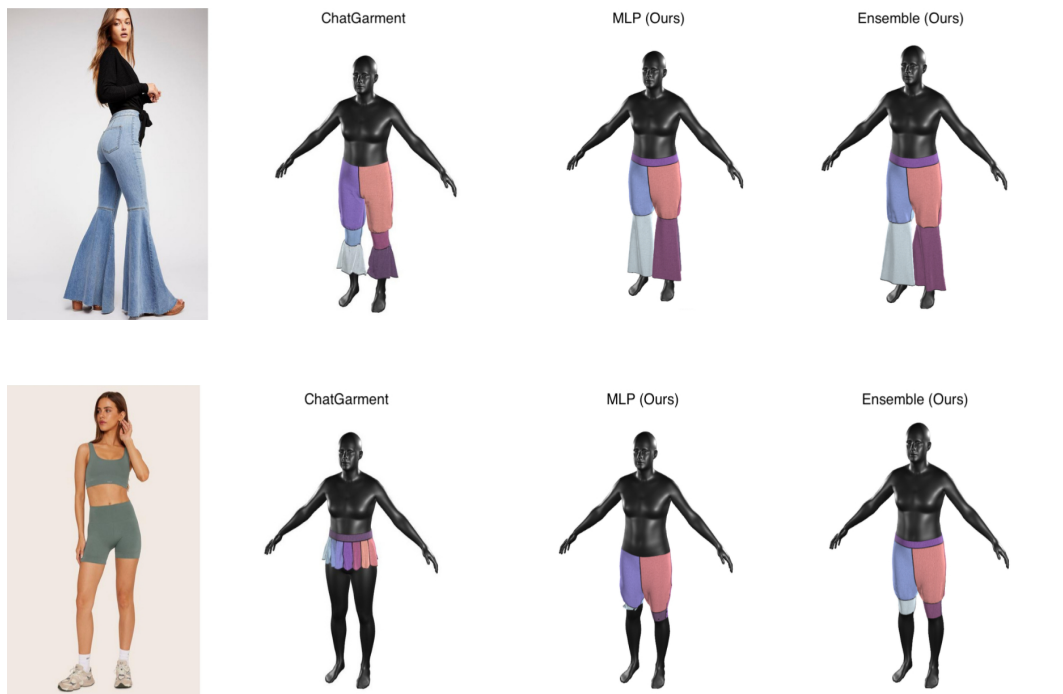


Figure 3: Qualitative comparisons of garment-reconstruction predictions.

5.2 Editing Garments

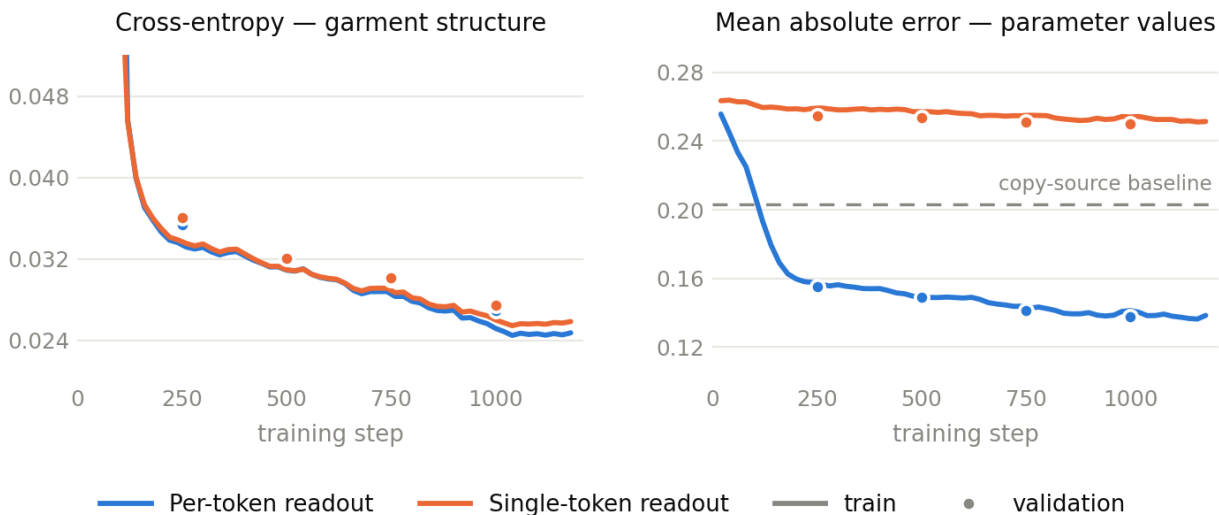
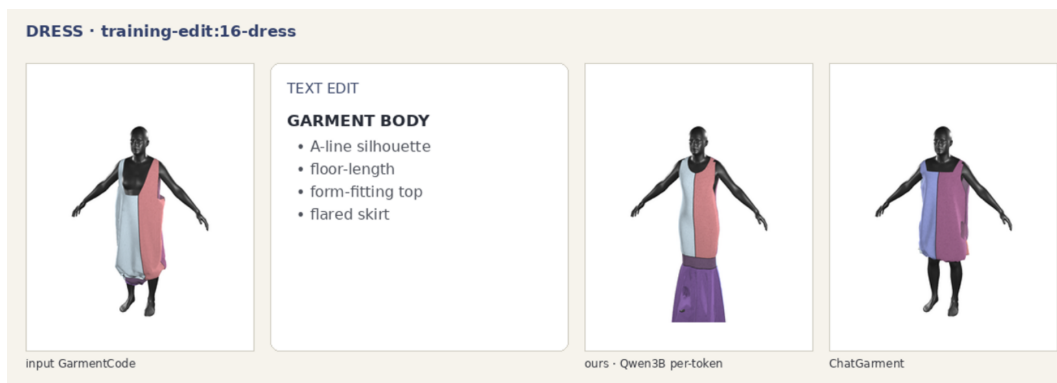


Figure 4: Garment-editing training and validation losses for single-token and per-token readouts.

Both variants learn the garment structure similarly, with nearly identical cross-entropy curves and only a small advantage for the per-token model. The main difference appears in continuous-value prediction: the per-token readout reduces validation MAE to about 0.14, whereas the single-token variant remains near 0.25 and performs worse than the copy-source baseline. This suggests that predicting each value from its own key-conditioned `<VAL>` representation is substantially more effective than regressing all 76 values from a single hidden state.

The following are comparisons of our predictions with ChatGarment’s predictions for several garment-editing prompts.



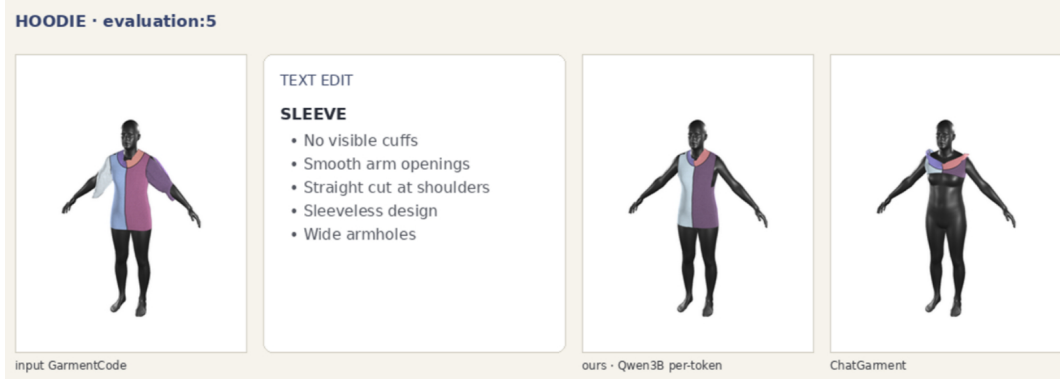


Figure 5: Qualitative comparisons of garment-editing predictions.

6 Conclusion, Limitation and Future Work

We presented a lightweight pipeline for garment reconstruction from a single RGB image. We used DinoV2 as a frozen encoder to generate image encoding for our MLP and Ensemble models which then generated GarmentCode parameters for 3D garment generation. Additionally, we also provide a framework for garment editing via natural language by fine-tuning Qwen2.5-3B on GarmentCode parameters + text input and edited GarmentCode parameters output pairs. Finally, our full framework (garment reconstruction + editing) uses 0.2 times (60M against 300M) of ChatGarment’s parameters and is able to give comparable and sometimes even better results. Although our models are able to give competitive results, they were trained mainly on synthetic data, including only 600 garments from our balanced dataset, which limits garment, appearance, and image diversity and may reduce generalisation to in-the-wild photographs.

Since GarmentCode may be limited in the types of shapes it can produce, we could use a GNN-based model [6] to predict edge lengths and vertex displacements to remesh and warp the existing topology to fit high-resolution outputs from 3D model-generation methods.

References

- [1] Siyuan Bian, Chenghao Xu, Yuliang Xiu, Artur Grigorev, Zhen Liu, Cewu Lu, Michael J Black, and Yao Feng. Chatgarment: Garment estimation, generation and editing via large language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025.
- [2] Maria Korosteleva and Sung-Hee Lee. Neuraltailor: Reconstructing sewing pattern structures from 3d point clouds of garments. *ACM Trans. Graph.*, 41(4), 2022.
- [3] Maria Korosteleva and Olga Sorkine-Hornung. GarmentCode: Programming parametric sewing patterns. *ACM Transaction on Graphics*, 42(6), 2023. SIGGRAPH ASIA 2023 issue.

- [4] Lijuan Liu, Xiangyu Xu, Zhijie Lin, Jiabin Liang, and Shuicheng Yan. Towards garment sewing pattern reconstruction from a single image. *ACM Transactions on Graphics (SIGGRAPH Asia)*, 2023.
- [5] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khaldov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. Dinov2: Learning robust visual features without supervision, 2024.
- [6] Tobias Pfaff, Meire Fortunato, Alvaro Sanchez-Gonzalez, and Peter W. Battaglia. Learning mesh-based simulation with graph networks. In *International Conference on Learning Representations*, 2021.
- [7] Joachim Tesch, Giorgio Becherini, Prerana Achar, Anastasios Yiannakidis, Muhammed Kocabas, Priyanka Patel, and Michael J. Black. BEDLAM2.0: Synthetic humans and cameras in motion. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2025.